

CODECRAFTERS - AN ONLINE CODE EDITOR PLATFORM USING MERN STACK

Nimit garg

21bcs4626@cuchd.in

Kirtan Nahar

21bcs8762@cuch.in

Akshay Mann

21BCS10215@cuchd.in

Hardik Jain

21BCS10861@cuchd.in

Department of Computer Science and Engineering Apex Institute of Technology

Chandigarh University Mohali, Punjab, India

Abstract—

In the rapidly expanding realm of the Internet, a multitude of applications originally confined to desktop environments are now migrating to the web, offering users ubiquitous access.

Developers, essential contributors to this digital transformation, require effective tools for application creation. This research endeavors to design and implement a real-time code editor application utilizing web socket technology, enhancing collaborative work on projects.

The application, named Code-Crafters, enables users to collaborate seamlessly on projects in real-time. The research methodology includes an in-depth analysis of existing code editor applications, the distribution of questionnaires, and an extensive review of relevant literature.

Code-Crafters, as a web application, furnishes a workspace where users can write, execute, and display code results through an integrated terminal. Notably, it facilitates real-time collaboration, allowing multiple users to work together on a project simultaneously.

The application also incorporates a chat feature to further enhance communication among collaborators. A distinctive feature of Code-Crafters is its built-in terminal, providing users with a comprehensive environment for code execution and project building. Moreover, Code-Crafters supports programming languages such as C, C++, and Java, making it versatile and accommodating for a range of developers. In

essence, Code-Crafters represents a holistic solution for developers seeking a collaborative, real-time code editing experience in an accessible online environment.

I. INTRODUCTION

A. Background

The trajectory of technological advancements in software engineering has witnessed a notable shift from desktop-based software design to web-based solutions. While established Integrated Development Environment (IDE) applications like Eclipse and Visual Studio have thrived, desktop-centric IDEs face inherent challenges, including prolonged configuration times and the need for installing specific plugins to run projects. This becomes especially cumbersome when dealing with multiple devices that necessitate individual configurations.

A paradigm shift is evident in the prevalence of software applications operating in the cloud, leveraging web browsers as interfaces. This approach ensures ubiquitous access, facilitates instant collaboration, and eliminates the need for desktop installations and configurations. One key technology employed for real-time collaboration is the adoption of a single IDE, exemplified by pair programming. Pair programming involves two programmers jointly accessing and working on the same code within a

shared development environment. This collaborative practice enables programmers to concurrently create, edit, and delete source code in real-time, addressing synchronization issues and allowing immediate visibility of code changes among collaborators.

The significance of collaborative technologies lies in their ability to facilitate teamwork among programmers, whether in debugging or discussing code, even when situated in different geographical locations. Consequently, there is a compelling need for an application that enhances programming efficiency, offering features such as real-time collaboration and the capability to create, execute, and display program results using an integrated terminal.

B. Problem Statement

Several problems plague the state of online code editors today:

Limited Feature Set: Numerous online code editors are missing crucial functions like real-time collaboration, sophisticated debugging tools, and multi-language compatibility.

Poor User Experience: Users frequently experience awkward interfaces, sluggish reaction times, and a lack of personalization options, which causes dissatisfaction and lower productivity.

Ineffective Collaboration: The creation of group projects is hampered by the inability of current platforms to provide collaborative coding among teams.

Compatibility Problems: Users that depend on a variety of environments may encounter difficulties using some online code editors because they are incompatible with specific browsers, operating systems, or devices.

C. Objectives

The goal of the online code editor project is to create a web-based environment that makes it simple and convenient to write, edit, and collaborate on code in a variety of programming languages. The platform will provide an intuitive user interface, real-time collaboration tools, and support for a variety of

programming languages and frameworks to developers, students, and instructors.

Editing Code

1. Code completion and syntax highlighting for a variety of programming languages.

2. For easier code reading, use line numbers and indentation.

3. A user-friendly interface with themes and layouts that are adjustable.

In-Motion Cooperation

4. Support for real-time code debugging and editing by many users.

5. Functionality for comments and chat for efficient collaboration.

Language Assistance

6. A full range of support for well-known programming languages like Python, JavaScript, C++, and more.

7. Compilers and interpreters for each language are integrated to provide real-time feedback on code execution.

D. System Description

A reliable and well-equipped hardware infrastructure is necessary for the effective development and operation of the online code editor. To guarantee the best performance and user experience, the following hardware requirements are advised:

Infrastructure for servers:

Processor: A dual-core or better CPU with a minimum clock speed of 2.5 GHz is required.

RAM: For smooth concurrent user interactions, at least 8 GB of RAM is required.

SSD storage with at least 256 GB of space is required for fast file access.

Network: For smooth data transfer, choose a gigabit Ethernet connection or higher.

Utilize load balancing to split up incoming traffic among various server instances.

By ensuring an equitable allocation of user requests, this lowers the possibility of server overload.

Processor: A CPU with four cores or more.

RAM: 16 GB of RAM is the bare minimum for effective data handling.

Storage: An SSD with enough room for project files and user data.

Depending on the needs of the project, use a relational or NoSQL database for your database management system.

II. LITERATURE REVIEW

The trajectory of technological progress in software engineering indicates a shift from desktop-oriented software design to web-based solutions. While numerous Integrated Development Environment (IDE) applications such as Eclipse and Visual Studio have been developed, desktop-based IDEs still face significant drawbacks, notably prolonged configuration times and the need for plugin installations to run projects. This becomes a considerable time investment, particularly when configuring numerous devices.

Cloud-based software applications, utilizing web browsers as user interfaces, have become prevalent. This approach ensures widespread accessibility, enables instant collaboration, and eliminates the need for installations and configurations on desktop computers. An example of technology facilitating immediate collaboration is the deployment of a single IDE, exemplified by pair programming. This practice involves two programmers accessing and working on the same code within a shared development environment. In pair programming, real-time capabilities empower programmers to concurrently create, edit, and delete source code, resolving synchronization issues. Changes made by one programmer are promptly visible to others working on the same project.

Collaborative technologies play a pivotal role in fostering teamwork among programmers, allowing them to collaborate on code fixes or program discussions within a shared environment, irrespective of geographical distances. Consequently, there is a pressing need for an application designed to enhance programming efficiency, featuring real-time collaboration and the ability to create, execute, and display program results using an integrated terminal.

III. EXISTING SYSTEM

Your project synopsis for an online code editor should include a description of the existing systems, platforms, or tools that are currently on the market in the "Existing System" section. The strengths, limitations, and gaps in the current systems are highlighted in this section to assist provide the context for your proposal. Here is an example of how to format this paragraph:

Overview of Current Systems

Introduce the reader to the state of online code editors right away. Mention how important they are to collaboration and modern software development.

Describe the present solutions:

Give a succinct rundown of some of the most popular online code editors now in use. Mention the brands, platforms, and fundamental features of each.

Emphasize the advantages and qualities of these systems, such as their simplicity of use, ability to support several programming languages, and real-time collaboration.

Specifications and Drawbacks:

List the main features that the current solutions offer, such as debugging tools, version control integration, and code highlighting.

Talk about the restrictions or drawbacks of these systems, such as the lack of support for some languages, restricted customization choices, or performance concerns.

IV. PROPOSED SYSTEM

The features and functionalities you intend to include in your online code editor are described in the Proposed System section of your project summary. Here is an example of how to format this paragraph:

1. Online code editor is the suggested system.

Users must first register and be authenticated. Create a user registration system that enables account creation for users. Give users access to authentication tools to protect their accounts and data.

2. The interface for editing code:

Create a user-friendly code editing interface that supports code completion, syntax highlighting, and error highlighting. Enable programmable themes to improve the appearance.

3. Support for Multiple Languages:

Give the editor the ability to support a variety of programming languages. To help developers write code more effectively, offer libraries and features specific to each language.

4. Real-time Teamwork:

Include real-time collaboration tools that enable several people to change the same code at the same time. Use version control to keep track of changes and make it simple for team members to communicate.

5. Testing and Code Execution:

Add a code execution environment so users may test and run their code right in the editor. Give fast feedback by displaying execution outcomes and error notifications.

6. Automatic Saving and History:

Use the auto-save feature to avoid data loss. Keep track of code revisions so that users can go back to earlier iterations if necessary.

7. Sharing and Embedding of Code:

Give users the option to distribute their projects or code snippets using individual URLs, enabling smooth code embedding into web pages or blogs.

8. Integration of documentation

To help users along their coding journey, integrate documentation resources, such as language references and tutorials.

9. User preferences and profiles:

Permit users to establish up profiles, alter their preferences, and preserve changes. Give each customer a customized experience based on their specific demands.

1. Performance Optimisation:

Improve the editor's efficiency to guarantee seamless code editing, even for bigger projects. Implement functions like effective memory management and asynchronous loading.

2. Cross-Platform Compatibility:

Ensure that the editor is usable on many platforms and browsers, such as PCs, tablets, and smartphones.

3. Security Measures:

Put security measures in place to safeguard user information and stop unauthorized access.

- Patch the system frequently to fix any potential vulnerabilities.

4. User Support and Feedback:

- Provide FAQs and instructions on how to use the editor's capabilities in the help area.

- Give users a way to voice their opinions and report problems.

14. Monetization Strategy - Describe your ideas for monetization, including any premium services or subscription-based business models, if any.

V. METHODOLOGY

1. Project Design and Planning:

Describe the overall goals and objectives of the online code editor and discuss the design considerations, including the choice of the MERN stack for the project.

2. System Architecture:

Provide an overview of the MERN stack architecture and how it is implemented in the online code editor. Explain the role of MongoDB for data storage, Express.js for server-side logic, React for the user interface, and Node.js for server-side runtime.

3. Database Design:

Outline the database schema and data model used for storing user projects and related information and discuss considerations for data retrieval, storage, and indexing in MongoDB.

4. Frontend Implementation:

Detail the structure and components of the React-based user interface and explain the design choices for the user experience and interface responsiveness. Discuss the integration of HTML, CSS, and JavaScript editors and how they interact with the backend.

5. Backend Implementation:

Describe the role of Express.js and Node.js in handling server-side logic and explain how the backend manages user authentication, project creation, storage, and retrieval.

6. Real-Time Collaboration Features:

If applicable, discuss the implementation of real-time collaboration features using technologies like WebSockets and explain how multiple users can simultaneously edit and collaborate on projects in real-time.

7. Testing:

Detail the testing strategy for the online code editor, including unit testing, integration testing, and

end-to-end testing and discuss how different components of the MERN stack are tested individually and as an integrated system.

8. Deployment:

Provide information on the deployment process, including the hosting environment and server configuration and discuss any challenges encountered during deployment and how they were addressed.

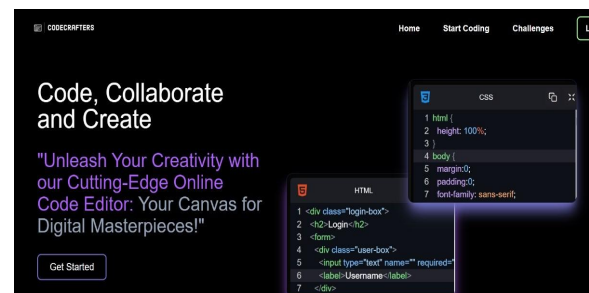
9. User Feedback and Iterative Development:

Explain how user feedback was collected during the development process and describe any iterations or improvements made based on user feedback.

VI. RESULT AND IMPLEMENTATION

4.1 Implementation of Solution

In the implementation phase of our online code editor project, we leverage modern tools and technologies for different aspects of the development. These encompass analysis, design, report preparation, project management, communication, as well as testing, characterization, interpretation, and data validation.



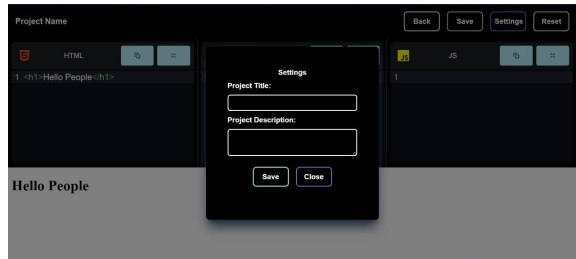
4.1.1 Code Editor Component

We have developed a code editor component in React that supports multiple programming languages, including HTML, CSS, and JavaScript.

This code editor component is designed with the following features:

Multi-Language Support: It supports HTML, CSS, and JavaScript, with syntax highlighting and code validation for each language.

User-Friendly Interface: Users can easily switch between full-screen and normal mode, copy code to the clipboard, and receive alerts for successful copying.



4.1.2 Analysis and Design

The design of the code editor component incorporates modern front-end development libraries and best practices, including React for building dynamic user interfaces and CodeMirror for code editing capabilities.

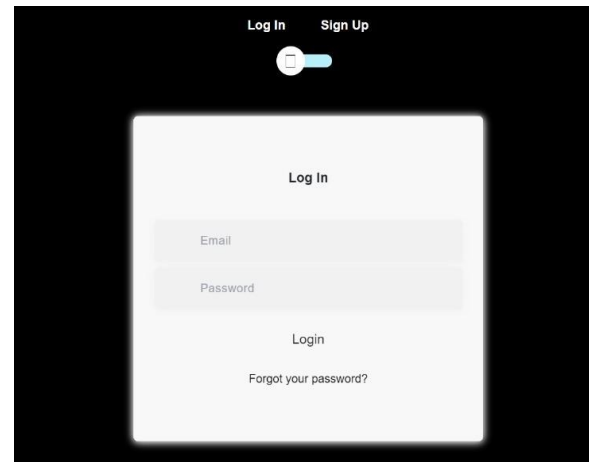


4.1.3 Project Management and Communication

Our project management and communication are facilitated by version control systems (e.g., Git and GitHub) to track changes, collaborate on the codebase, and manage project tasks.

4.1.4 Authentication

We ensure the robustness and reliability of our code editor by implementing testing strategies, including unit tests, integration tests, and user acceptance testing. Additionally, data validation mechanisms are in place to prevent malicious code injections



VII. CONCLUSION

The online code editor project has been developed with the goal of providing a versatile and user-friendly platform for coding in HTML, CSS, and JavaScript. Throughout the development process, we set certain expectations and aimed for specific outcomes. Here, we summarize the expected results, any deviations from those expectations, and the reasons for these deviations.

User-Friendly Interface: The project aimed to deliver an intuitive and user-friendly code editing experience for developers working with HTML, CSS, and JavaScript.

Code Highlighting: We expected the code editor to provide syntax highlighting for different programming languages, enhancing code readability.

Code Execution: The project was designed to allow users to execute and visualize the code output directly within the application.

Copy-to-Clipboard Functionality: A feature to copy code snippets to the clipboard was intended for convenience.

VIII. FUTURE WORK

Moving forward, there are several avenues for improvement and extension of the online code editor project. The future work involves the following areas:

Security Enhancements: To address the code execution limitations, future work should focus on implementing robust security measures to enable safer real-time code execution within the editor.

Additional Language Support: Expanding the supported programming languages and frameworks, such as Python, Ruby, and various front-end libraries, would enhance the editor's versatility.

Enhanced User Interface: Continuously refining

and improving the user interface to provide a seamless coding experience, including additional customization options and themes.

Version Control Integration: Integration with version control systems like Git for collaborative coding and code version tracking.

Error Handling and Debugging: Implementing effective error handling and debugging features to assist users in identifying and resolving code issues.

Collaborative Features: Enabling real-time collaborative coding features to allow multiple users to work on code together

REFERENCES

- [1] M. Doernhoefer, "Surfing the Net for Software Engineering Notes", ACM SIGSOFT Software Engineering Notes, Vol. 24, No. 3, (1999), pp. 15–24.
- [2] L. C. L. Kats, R. G. Vogelij, K. T. Kalleberg, and E. Visser, "Software development environments on the web", in Proceedings of the ACM international symposium on New ideas, new paradigms, and reflections on programming and software - Onward! '12, (2012), pp. 99.
- [3] M. Goldman, "Role-based interfaces for collaborative software development", in Proceedings of the 24th Annual ACM Symposium Adjunct on User Interface Software and Technology - UIST '11 Adjunct, (2011), pp. 23.
- [4] F. Fröblier, "A Practice Theoretical Analysis of Real Time Collaboration Technology: Skype and Sametime in Software Development Projects", Göttingen: Cuvillier, (2008).
- [5] S. Klein, N. Vehring, and M. Kramer, "Introducing Real Time Communication: Frames, Modes & Rules", in Proceedings 23rd Bled eConference eTrust: Implications for the Individual, (2010), pp. 591–606.
- [6] K. Riemer and F. Fröblier, "Introducing Real-Time Collaboration Systems: Development of a Conceptual Scheme and Research Directions", Communications of the Association for Informations Systems (CAIS), Vol. 20, (2007), pp. 204–225.
- [7] M. Goldman, G. Little, and R. C. Miller, "Real-time Collaborative Coding in a Web IDE", in Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology, (2011), pp. 155–164.
- [8] H. Fan, C. Sun, and H. Shen, "ATCoPE: Any-time Collaborative Programming Environment for Seamless Integration of Real-time and Non-real-time Teamwork in Software Development", in Proceedings of the 17th ACM International Conference on Supporting Group Work, (2012), pp. 107–116.
- [9] H. Bani-Salameh, C. Jeffery, Z. Al-Sharif, and I. Abu Doush, "Integrating Collaborative Program Development and Debugging within a Virtual Environment", in Proceedings of the 14th Collaboration Researchers' International Workshop on Groupware, Vol. 5411, (2008), pp. 107–120.